KUBERNETES TRAINING

Module 5:

Cluster Scaling, Node Maintenance, DaemonSets, Jobs & CronJobs

Operate the cluster: add capacity, safely maintain nodes, run one-per-node agents, and schedule batch work.

2026 MODERNIZED EDITION

From workload management to node operations

Modules 1–2

Containers, Kubernetes architecture, kubectl

Module 3

Pods + Services

Module 4 / 4B

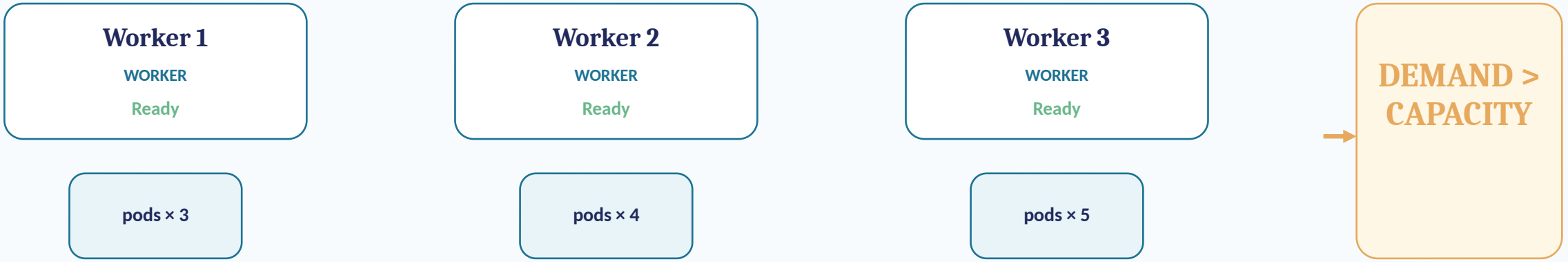
Namespaces, Deployments, Service types + rollout strategies

TODAY

1. Scale out with kubeadm join
2. **Cordon → drain → maintenance → uncordon**
3. DaemonSets: one Pod per eligible node
4. Jobs: run-to-completion + retries + parallelism
5. CronJobs: recurring schedules + cleanup
6. Hands-on operations + troubleshooting

Why scale a cluster?

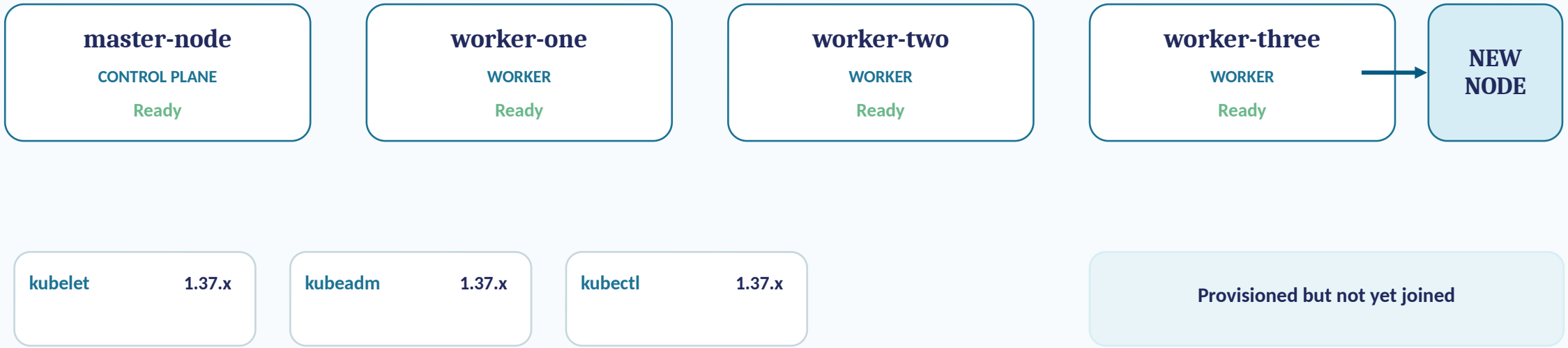
When demand grows beyond the capacity of the current worker pool, add nodes rather than forcing more workloads onto the same machines.



Scale-out adds scheduling capacity to the cluster. Kubernetes then has another node on which to place eligible Pods.

Step 1 — provision the new worker node

Keep the new host compatible with the cluster: same Kubernetes minor version, a container runtime, swap handled.



Step 2 — generate a join token

$ Terminal

```
kubeadm token create --print-join-command
```

OUTPUT

```
kubeadm join 10.0.0.10:6443 \
  --token abcdef.0123456789abcdef \
  --discovery-token-ca-cert-hash \
  sha256:9f1c...e8a2
```

What the command gives you

A complete worker-node join command: API endpoint + bootstrap token + CA public-key hash for discovery trust.

Token lifetime

The kubeadm-created bootstrap token has a default 24-hour TTL. If it expires, create a new token and print a fresh join command.

Step 3 — join the worker, then verify it

$ Terminal

```
kubeadm join 10.0.0.10:6443 \
  --token abcdef.0123456789abcdef \
  --discovery-token-ca-cert-hash sha256:9f1c...e8a2
```

OUTPUT

```
[preflight] Running pre-flight checks
[kubelet-start] Starting the kubelet
[kubelet-check] The kubelet is healthy
This node has joined the cluster:
* Certificate signing request was sent
  to apiserver and a response was received.
```

$ Terminal

```
kubectl get nodes
```

OUTPUT

```
NAME           STATUS   ROLES           AGE   VERSION
master-node    Ready    control-plane   12d   v1.37.0
worker-one     Ready    <none>          12d   v1.37.0
worker-two     Ready    <none>          12d   v1.37.0
worker-three   Ready    <none>          18s   v1.37.0
```

The new node is now schedulable capacity — subject to labels, taints, affinity and resource availability.

Practical note — join tokens expire

24-hour default

kubeadm init creates an initial bootstrap token with a default TTL of 24 hours.

Need to join later?

Generate a new token on the control-plane host and print a fresh join command.

$ Terminal

```
kubeadm token create --print-join-
command
```

Security habit

Treat bootstrap tokens as temporary credentials. Do not paste real production tokens into training material.

Why node maintenance needs a workflow

Upgrade

kernel / kubelet

Repair

hardware / disk

Replace

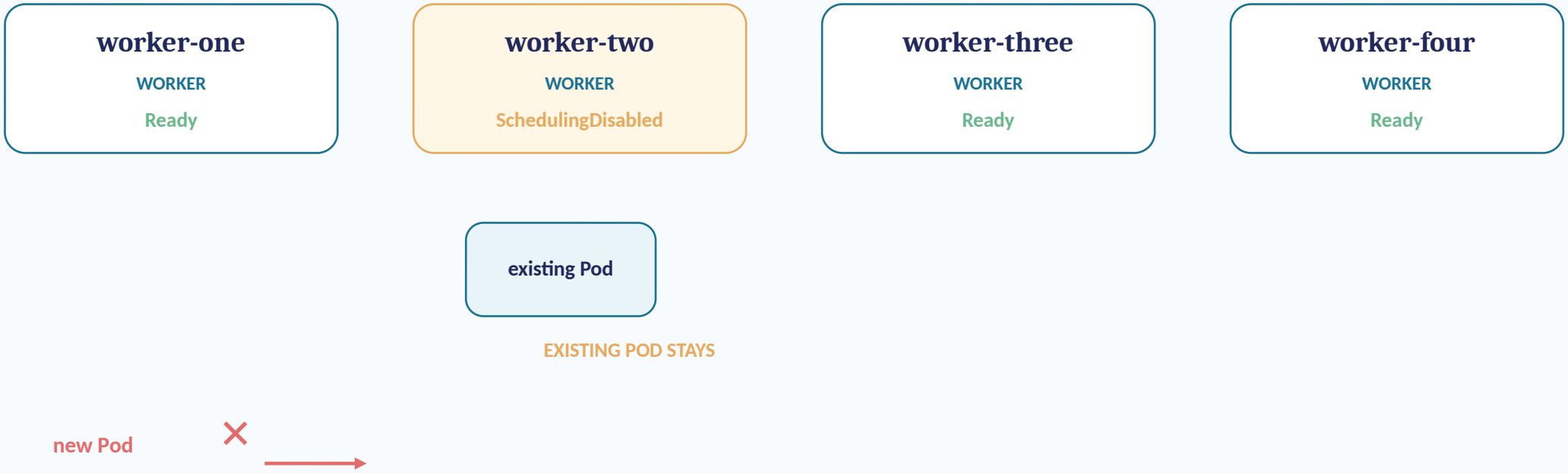
cloud VM / host

Observe

move workloads safely

Goal: make the node unavailable for scheduling, move evictable workloads safely, perform maintenance, then restore scheduling.

Cordon — stop new scheduling, keep existing Pods



Cordon changes the scheduler decision. It does not evict the Pods already running there.

Cordon in action

$ Terminal

```
kubectl cordon worker-two
```

OUTPUT

```
node/worker-two cordoned
```

$ Terminal

```
kubectl get nodes
```

OUTPUT

```
NAME          STATUS                     ROLES    AGE   VERSION
worker-one    Ready                      <none>   12d   v1.37.0
worker-two    Ready,SchedulingDisabled    <none>   12d   v1.37.0
worker-three  Ready                      <none>   12d   v1.37.0
```

SchedulingDisabled is the visible signal: existing workloads are untouched; new scheduling avoids the node.

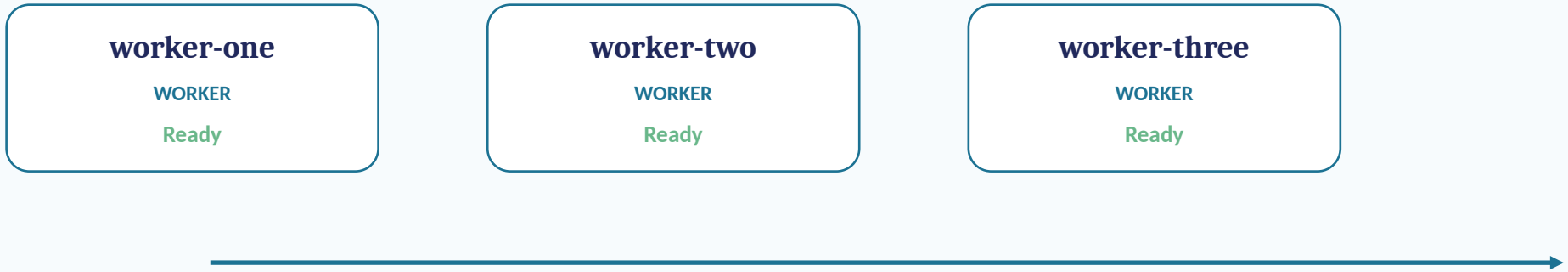
Uncordon — return the node to normal scheduling

$ Terminal

```
kubectl uncordon worker-two
```

OUTPUT

```
node/worker-two uncordoned
```



After maintenance: uncordon does not “restart” Pods. It simply makes the node eligible for new scheduling again.

Cordon vs Drain — the key difference

CORDON

Stops NEW scheduling

1. Scheduling
2. **No eviction**
3. Node remains active

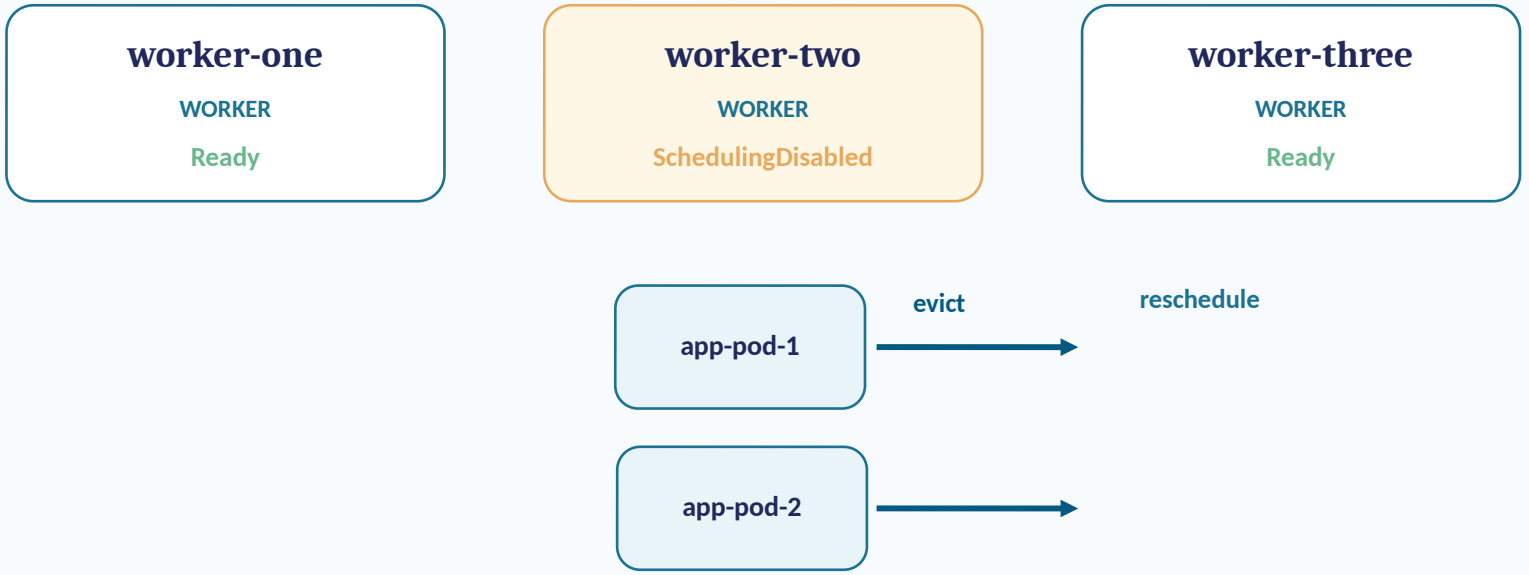
DRAIN

Evicts existing workloads

1. Scheduling
2. **Eviction**
3. Workloads leave

Think: cordon changes WHERE new Pods may go. Drain actively removes eligible Pods from the node.

Drain — evict workloads before maintenance




```
$ Terminal

kubectl drain worker-two --ignore-daemonsets
```

Drain waits for graceful termination and respects eviction behavior such as PodDisruptionBudgets.



Drain with DaemonSets — the special case

$ Terminal

```
kubectl drain worker-two
```

OUTPUT

```
error: unable to drain node "worker-two"
due to error: cannot delete DaemonSet-
managed Pods (use --ignore-daemonsets
to ignore): kube-system/kube-proxy-x7k2p
```

$ Terminal

```
kubectl drain worker-two --ignore-daemonsets
```

OUTPUT

```
Warning: ignoring DaemonSet-managed Pods: …
evicting pod default/app-pod-1
pod/app-pod-1 evicted
node/worker-two drained
```

Why? DaemonSet Pods are node-local by design and Kubernetes does not evict them during drain by default.

Full maintenance workflow

1 CORDON

$ Terminal

```
kubectl cordon worker-two
```

No new scheduling

2 DRAIN

$ Terminal

```
kubectl drain worker-two
--ignore-daemonsets
```

Evict eligible Pods

3 MAINTAIN

$ Terminal

```
upgrade / repair / replace
```

Node is out of service

4 UNCORDON

$ Terminal

```
kubectl uncordon worker-
two
```

Scheduling resumes

Operational rule: do not power down / delete the node until drain completes successfully.

PodDisruptionBudget — how many Pods may be down

What it is

A PodDisruptionBudget (PDB) is a rule you attach to an app:

"During planned maintenance, never take down more than N of my Pods at the same time."

kubectl drain does not just delete Pods. It asks the API server for permission to evict each one. The API server checks the PDB. If one more eviction would break the rule, it says "not yet" and drain waits and retries.

web-pdb.yaml

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  maxUnavailable: 1        # or: minAvailable: 2
  selector:
    matchLabels:
      app: web
```

PDB protects (voluntary)

kubectl drain • node upgrade • Cluster Autoscaler scale-down • anything that uses the eviction API

PDB cannot help (involuntary)

Node crash • hardware or network failure • OOM kill • kubectl delete pod (not an eviction)

Use ONE of the two settings (a number or a percent)

maxUnavailable: 1 → at most 1 Pod may be down. Good for most apps.

minAvailable: 2 → at least 2 Pods must stay Ready.

Check it: kubectl get pdb → ALLOWED DISRUPTIONS column.

Example: web has 3 Pods and web-pdb says maxUnavailable: 1

1. Drain asks to evict Pod A → allowed (0 Pods are down) → A is evicted, ALLOWED DISRUPTIONS drops to 0.
2. Drain asks to evict Pod B → "not yet", because A is still down. Drain waits and retries every few seconds.
3. A's replacement becomes Ready on another node → ALLOWED DISRUPTIONS is back to 1 → Pod B is evicted. Repeat. The app never drops below 2 Pods.

Trap: maxUnavailable: 0 (or minAvailable = replicas) allows 0 disruptions, so drain waits forever. See the labs on slides 39–40.

Hands-on — observe scheduling, cordon, drain, restore

1 Pin a test Pod to worker-one

```
$ Terminal
kubectl apply -f test-pod-1.yaml
kubectl get pods -o wide
```

test-pod-1 has nodeSelector: worker-one

2 Cordon worker-one

```
$ Terminal
kubectl cordon worker-one
kubectl get nodes
```

worker-one: Ready,SchedulingDisabled

3 Try another Pod

```
$ Terminal
kubectl apply -f test-pod-2.yaml
kubectl get pods -o wide
```

Same nodeSelector → Pending (unschedulable)

4 Drain the node

```
$ Terminal
kubectl drain worker-one \
  --ignore-daemonsets --force
kubectl get pods -o wide
```

Bare Pods are deleted for good (--force)

5 Restore scheduling

```
$ Terminal
kubectl uncordon worker-one
kubectl get pods -o wide
```

test-pod-2 leaves Pending and starts

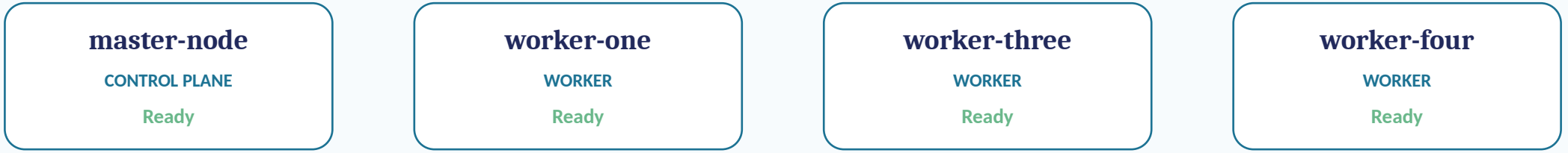
⚠ Permanent removal — delete the Node object

$ Terminal

```
kubectl delete node worker-two
```

OUTPUT

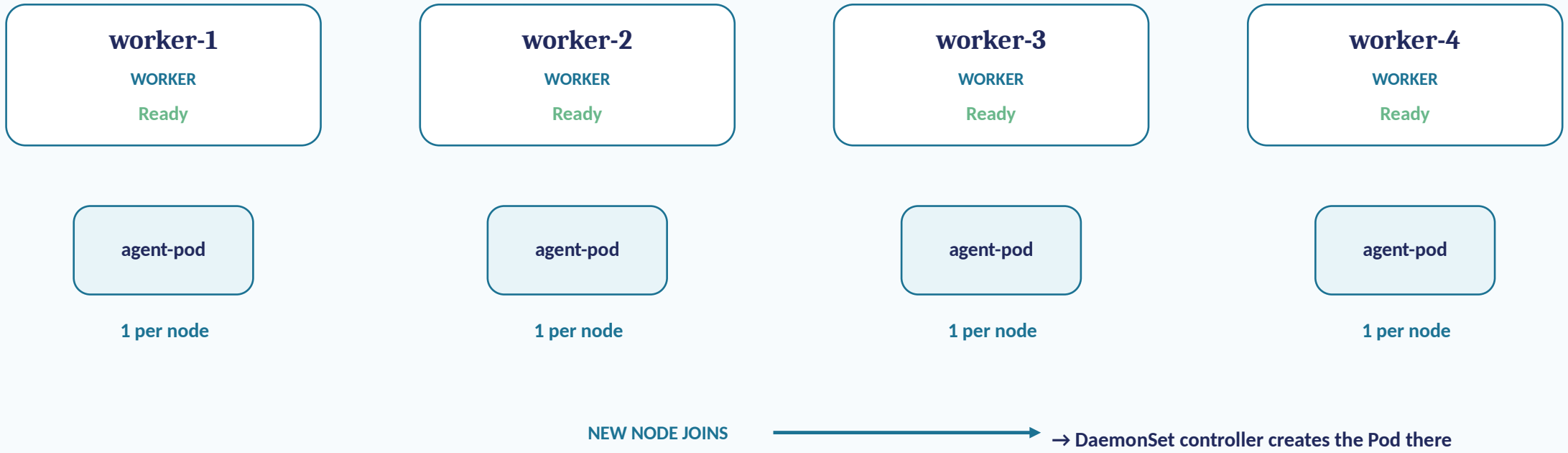
```
node "worker-two" deleted
```



Use this only for permanent removal. Drain first; deleting the API object does not perform the physical machine cleanup for you.

What is a DaemonSet?

A DaemonSet ensures a copy of a Pod runs on every eligible node — automatically extending coverage when nodes join.



When to use a DaemonSet

Logging

Fluentd / Filebeat

One agent per node reads node-local logs.

Monitoring

Prometheus node exporter

One exporter exposes node-level metrics.

Networking

CNI plugins

Node-local networking components run where needed.

Security

Node agents

Security / runtime agents can cover every eligible node.

Mental model: Deployment manages an application replica set; DaemonSet expresses node-local coverage.

DaemonSet YAML — full runnable example

daemonset.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: nginx-daemonset
  labels:
    app: nginx
spec:
  selector:
    matchLabels:
      name: nginx
  template:
    metadata:
      labels:
        name: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.28-alpine
          ports:
            - containerPort: 80
```

Why the selector matters

The DaemonSet selector must match the Pod template labels. That relationship identifies the Pods it owns.

What you get

One Pod on each eligible node. Control-plane nodes are skipped unless the Pod tolerates their taint. New nodes get a Pod automatically.

Apply and inspect a DaemonSet

$ Terminal

```
kubectl apply -f daemonset.yaml
kubectl get daemonsets
```

OUTPUT

```
NAME              DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
nginx-daemonset   4         4         4       4            4           <none>          25s
```

$ Terminal

```
kubectl get pods -o wide
```

OUTPUT

```
NAME                     READY   STATUS    NODE
nginx-daemonset-a1b2c    1/1     Running   worker-one
nginx-daemonset-d3e4f    1/1     Running   worker-two
nginx-daemonset-g5h6i    1/1     Running   worker-three
nginx-daemonset-j7k8l    1/1     Running   worker-four
```

Restrict a DaemonSet to specific nodes

$ Terminal

```
kubectl label node worker-one monitoring=enabled

kubectl patch daemonset nginx-daemonset -p '
{"spec":{"template":{"spec":{"nodeSelector":
 {"monitoring":"enabled"}}}}}'
```

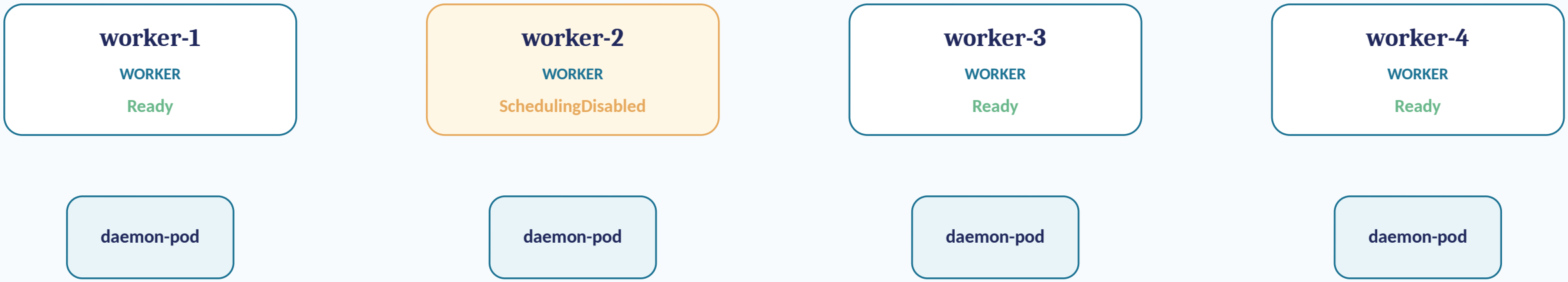
nodeSelector

`monitoring=enabled`

Only nodes with this label run the DaemonSet Pod. Label a node → Pod appears; remove the label → Pod goes.

Select nodes by a label that describes intent (monitoring=enabled, disk=ssd) — not by hostname, which changes when nodes are replaced.

DaemonSets + drain — why drain skips them



Drain evicts normal workload Pods...

...but DaemonSet Pods remain by design.

```
$ Terminal
kubectl drain worker-two --ignore-daemonsets
```

DaemonSets ensure one Pod per eligible node — removing it during drain would immediately trigger replacement.

Updating a DaemonSet

$ Terminal

```
kubectl set image daemonset/nginx-daemonset \
  nginx=nginx:1.29-alpine
kubectl rollout status daemonset/nginx-daemonset
```

OUTPUT

```
daemonset.apps/nginx-daemonset image updated
Waiting for daemon set "nginx-daemonset"
  rollout to finish: 1 out of 4 new pods …
daemon set "nginx-daemonset" successfully
  rolled out
```

What changes

Changing the Pod template replaces Pods node by node (RollingUpdate, maxUnavailable 1 by default). kubectl rollout history and undo work like for Deployments.

Verify

kubectl rollout history ds/nginx-daemonset
kubectl rollout undo ds/nginx-daemonset

What is a Job?

DEPLOYMENT

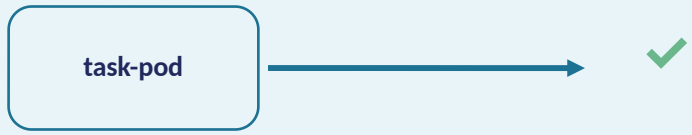
continuous service



keep replacing / scaling

JOB

run → complete → stop



batch / finite work

Job YAML — run a task and exit

job.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: simple-job
spec:
  template:
    spec:
      containers:
        - name: job-container
          image: busybox:1.37
          command: ["echo", "Hello from Kubernetes Job!"]
      restartPolicy: Never
```

Run-to-completion

The Job owns a Pod that exits after the command completes successfully.

Allowed restartPolicy

For Jobs, the Pod restartPolicy is `Never` or `OnFailure`.

Production extras

ttlSecondsAfterFinished and activeDeadlineSeconds

Runbook pp. 222–223 • simple Job manifest, modernized with pinned tag

Apply and verify a successful Job

$ Terminal

```
kubectl apply -f job.yaml
kubectl get jobs
```

OUTPUT

```
NAME         STATUS     COMPLETIONS   DURATION   AGE
simple-job   Complete   1/1           3s         8s
```

$ Terminal

```
kubectl get pods
kubectl logs job/simple-job
```

OUTPUT

```
NAME               READY   STATUS      AGE
simple-job-5krrn   0/1     Completed   6s

Hello from Kubernetes Job!
```

COMPLETIONS 1/1 is the success signal: the Job reached its requested successful completion count.

Job failures — retries and BackoffLimitExceeded

job-fail.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: fail-job
spec:
  backoffLimit: 3
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: task
          image: busybox:1.37
          command: ["sh", "-c", "exit 1"]
```

OUTPUT

```
NAME       STATUS   COMPLETIONS   DURATION
fail-job   Failed   0/1           70s

# 4 Pods: 1 try + 3 retries (back-off 10s, 20s, 40s)
Warning  BackoffLimitExceeded
  Job has reached the specified backoff limit
```

restartPolicy controls Pod retries

`Never` creates replacement Pods after failure; `OnFailure` can restart the container within the Pod.

backoffLimit controls Job failure

After the configured retry budget is exhausted, the Job becomes Failed and needs intervention.

Parallel Jobs — completions vs parallelism

parallel-job.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: parallel-job
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
      containers:
        - name: job-container
          image: busybox:1.37
          command: ["echo", "Running Parallel Job"]
      restartPolicy: Never
```



2 Pods max at once

Then the last one runs.

OUTPUT

```
NAME           STATUS     COMPLETIONS
parallel-job   Complete   3/3
```

Deleting a Job

$ Terminal

```
kubectl delete job parallel-job
```

OUTPUT

```
job.batch "parallel-job" deleted from
default namespace
```

Job → Pods

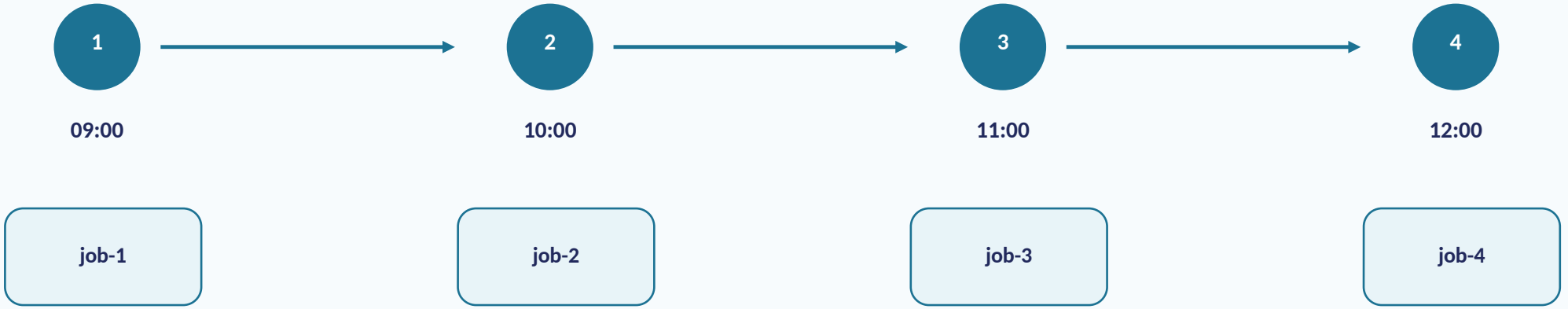
delete Job →

associated Pods are removed with the Job

Keep completed Jobs temporarily when you still need their logs or status; clean them up deliberately.

What is a CronJob?

A CronJob schedules Jobs at recurring times, using cron syntax.



Each scheduled run creates a Job; each Job creates a Pod that performs the work.

CronJob YAML — run every minute

cronjob.yaml

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: simple-cronjob
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: cron-container
              image: busybox:1.37
              command: ["echo", "Hello from Kubernetes CronJob!"]
          restartPolicy: Never
```

CRON LEGEND

MIN	*
HOUR	*
DAY	*
MONTH	*
WEEKDAY	*

Modern timezone control

Set .spec.timeZone (e.g. "Europe/Berlin") so 09:00 means 09:00 there — otherwise the controller's zone, often UTC, is used.

Apply and monitor a CronJob

$ Terminal

```
kubectl apply -f cronjob.yaml
kubectl get cronjobs
```

OUTPUT

```
NAME            SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE
simple-cronjob  */1 * * * *   <none>     False     0        22s
```

$ Terminal

```
kubectl get jobs
kubectl get pods -o wide
```

OUTPUT

```
NAME                      STATUS     COMPLETIONS   DURATION   AGE
simple-cronjob-29844351   Complete   1/1           3s         82s
simple-cronjob-29844352   Complete   1/1           3s         22s

# Job name = CronJob name + scheduled minute (Unix time / 60)
```

$ Terminal

```
kubectl logs job/simple-cronjob-29844352
```

```
Hello from Kubernetes CronJob!
```

CronJob behavior — runs, retries, overlap, history

NEW RUN

CronJob creates a new Job on each scheduled run.

RETRY

The Job handles Pod failure using its retry rules.

CONCURRENCY

Configure whether overlapping Jobs may run together.

HISTORY

Keep only the successful / failed Job history you need.

Operational note: CronJob schedule interpretation uses the controller time zone unless `.spec.timeZone` is explicitly set.

Hands-on — DaemonSet + drain + CronJob

1 Deploy DaemonSet

$ Terminal

```
kubectl apply -f daemonset.yaml
kubectl get pods -o wide
```

2 Cordon + drain

$ Terminal

```
kubectl cordon worker-two
kubectl drain worker-two --ignore-
daemonsets
```

3 Observe DaemonSet

$ Terminal

```
kubectl get pods -o wide
# daemon Pod remains on worker-two
```

4 Create CronJob

$ Terminal

```
kubectl apply -f cronjob.yaml
kubectl get cronjobs
```

5 Watch scheduled Jobs

$ Terminal

```
kubectl get jobs
kubectl get pods -o wide
kubectl logs job/<job-name>
```

6 Restore node

$ Terminal

```
kubectl uncordon worker-two
```

Lab: join a real 4th node with kubeadm

$ Terminal

```
docker run -d --name lab-worker3 --privileged \
  --network kind --cgroupns=private … \
  "$NODE_IMAGE"      # same image as lab-worker
```

OUTPUT

```
43f74e936f33
# a fresh machine: kubelet, kubeadm and
# containerd installed — not in the cluster yet
```

$ Terminal

```
JOIN_CMD=$(docker exec lab-control-plane \
  kubeadm token create --print-join-command)
docker exec lab-worker3 $JOIN_CMD
```

OUTPUT

```
[kubelet-check] The kubelet is healthy after 503ms
This node has joined the cluster:
* Certificate signing request was sent to apiserver …
```

$ Terminal

```
kubectl get nodes
kubectl get pods -n kube-system -o wide | grep worker3
```

OUTPUT

```
lab-worker3   Ready   <none>   13s   v1.37.0
kindnet-4fgrd      Running   lab-worker3
kube-proxy-dv9nn   Running   lab-worker3
```

Lab: what cordon really does

$ Terminal

```
kubectl cordon lab-worker
kubectl scale deploy web --replicas=9
```

OUTPUT

```
Pods per node after scaling:
2 lab-worker   ← only the old ones
4 lab-worker2   3 lab-worker3
```

$ Terminal

```
kubectl run pinned … nodeSelector:
  kubernetes.io/hostname: lab-worker
```

OUTPUT

```
pinned   0/1   Pending
FailedScheduling: … 1 node(s) were unschedulable
```

$ Terminal

```
kubectl run bare … nodeName: lab-worker
```

OUTPUT

```
bare   1/1   Running   lab-worker
# nodeName skips the scheduler — cordon can't stop it
```

Cordon adds the node.kubernetes.io/unschedulable:NoSchedule taint. The scheduler obeys it; nodeName bypasses the scheduler.

Lab: why drain refuses

$ Terminal

```
kubectl apply -f web-pdb-strict.yaml
# maxUnavailable: 0
```

OUTPUT

```
NAME      MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS
web-pdb   N/A             0                 0
```

$ Terminal

```
kubectl drain lab-worker
```

OUTPUT

```
error: unable to drain node "lab-worker" due to error:
  cannot delete Pods that declare no controller
    (use --force to override): default/bare,
  cannot delete DaemonSet-managed Pods
    (use --ignore-daemonsets to ignore):
    kube-system/kindnet-2b649, kube-system/kube-proxy-fnjlw
```

$ Terminal

```
kubectl drain lab-worker --ignore-daemonsets --force
```

OUTPUT

```
error when evicting pods/"web-…-dwjfs" (will retry after 5s):
Cannot evict pod as it would violate the pod's disruption budget.
```

Lab: a PDB that allows maintenance

$ Terminal

```
kubectl apply -f web-pdb.yaml
# maxUnavailable: 1
# unhealthyPodEvictionPolicy: AlwaysAllow
```

OUTPUT

```
NAME      MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED
web-pdb   N/A             1                 1
```

$ Terminal

```
kubectl drain lab-worker \
  --ignore-daemonsets --force
```

OUTPUT

```
pod/web-…-9n4pp evicted
… (will retry after 5s): … disruption budget.
pod/web-…-dwjfs evicted
node/lab-worker drained
```

$ Terminal

```
kubectl uncordon lab-worker
kubectl rollout restart deploy/web
```

OUTPUT

```
after uncordon:  0 / 5 / 4 Pods per node
after restart:   3 / 3 / 3 Pods per node
```

Lab: DaemonSet coverage

$ Terminal

```
kubectl apply -f daemonset.yaml
kubectl get ds node-agent
```

OUTPUT

```
NAME         DESIRED   CURRENT   READY
node-agent   2         2         2
# 2 workers — the control plane has a NoSchedule taint
```

$ Terminal

```
# add a toleration for
#   node-role.kubernetes.io/control-plane
kubectl apply -f daemonset-all.yaml
```

OUTPUT

```
node-agent   3         3         3
# now also on lab-control-plane
```

$ Terminal

```
kubectl label node lab-worker2 monitoring=enabled
# + nodeSelector: {monitoring: enabled}
```

OUTPUT

```
node-agent   1   1   1   monitoring=enabled
# label another node → a Pod appears there
```

Lab: Jobs that fail smart

$ Terminal

```
kubectl apply -f job-fail.yaml
# backoffLimit: 2
```

OUTPUT

```
fail-job   Failed   0/1   34s
# 3 Pods: 1 try + 2 retries (back-off 10 s, 20 s)
```

$ Terminal

```
kubectl apply -f job-policy.yaml
# podFailurePolicy:
#   exit code 42 → FailJob
```

OUTPUT

```
policy-job   Failed   0/1   3s

Failed=True PodFailurePolicy: Container task
for pod default/policy-job-q48sq failed with
exit code 42 matching FailJob rule at index 0

# 1 Pod, 3 seconds — no wasted retries
```

$ Terminal

```
kubectl apply -f job-indexed.yaml
```

OUTPUT

```
indexed-job   Complete   5/5   19s
processing shard 0 of 5 … shard 4 of 5
```

Lab: a production-ready CronJob

cronjob.yaml

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: report
spec:
  schedule: "*/1 * * * *"
  timeZone: "Etc/UTC"
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 30
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      backoffLimit: 2
      activeDeadlineSeconds: 120
      ttlSecondsAfterFinished: 3600
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: report
              image: busybox:1.37
              command: ["sh", "-c",
                "echo report at $(date -u +%T)"]
```

Why each field

timeZone → 09:00 means your 09:00
Forbid → never two runs at once
startingDeadline → skip late runs
history limits → tidy job list
activeDeadline → kill hung runs

Test it now

kubectl create job report-manual --from=cronjob/report runs it immediately. Pause with suspend: true.

every minute • UTC • no overlap

Batch & node operations in 2026

Jobs grew up — stable in v1.37

podFailurePolicy: fail fast on bugs (exit codes), ignore disruptions like drains.

backoffLimitPerIndex, successPolicy (Indexed Jobs), podReplacementPolicy and managedBy — all without alpha/beta markers in kubectl explain on v1.37.

CronJob timeZone has been stable since v1.27.

Drains that don't get stuck

PodDisruptionBudget unhealthyPodEvictionPolicy: AlwaysAllow lets a drain evict Pods that are Running but not Ready.

The default (IfHealthyBudget) can block node maintenance when an app is already broken — exactly when you need to move it.

2026 takeaway: podFailurePolicy on every Job, AlwaysAllow on every PDB, and drains become predictable.

Verified with kubectl explain on Kubernetes v1.37 • Job, CronJob and PodDisruptionBudget API reference

HorizontalPodAutoscaler — scale Pods on CPU

hpa-web.yaml (the HPA part)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hpa-web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hpa-web
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 30
```

How it decides

Every 15 s: desired = ceil(current × actual ÷ target).

Utilization is a % of the CPU request — no request, no scaling.

Needs metrics-server

kubectl top must work first. Managed clusters ship it; on kind you install it (Step 17).

1 → 5 Pods under load • back to 1

Lab: the HPA scales 1 → 5 → 1

$ Terminal

```
kubectl apply -f hpa-web.yaml
kubectl get hpa hpa-web
```

OUTPUT

```
TARGETS         REPLICAS
cpu: 0%/50%     1
```

$ Terminal

```
kubectl run load --image=busybox:1.37 …
  # 8 wget loops against http://hpa-web
kubectl get hpa hpa-web
```

OUTPUT

```
cpu: 500%/50%   5
New size: 5; reason: cpu … above target
```

$ Terminal

```
kubectl delete pod load --now
kubectl autoscale deployment noreq --cpu=50%
```

OUTPUT

```
New size: 1; reason: All metrics below target
noreq   cpu: <unknown>/50%  ← no request
```

Node autoscaling — Cluster Autoscaler vs Karpenter

Cluster Autoscaler

Watches for Pods stuck Pending because no node fits, and grows an existing node group (an AWS Auto Scaling group, a GKE or AKS node pool).

Removes nodes that stay under-used. Works on every major cloud — the long-time standard.

Karpenter

Also reacts to Pending Pods, but skips node groups: it picks an instance type that fits the waiting Pods and creates the machine directly.

Consolidates onto fewer, cheaper nodes. Powers EKS Auto Mode and AKS Node Auto-Provisioning.

HPA adds Pods → Pods go Pending → the node autoscaler adds nodes. Scale-down drains nodes and respects PDBs.

Not runnable on kind — there is no cloud to create machines • kubernetes/autoscaler • kubernetes-sigs/karpenter

Lab: DaemonSet surge — no gap on any node

$ Terminal

```
kubectl patch daemonset node-agent -p \
 '{"spec":{"minReadySeconds":5,
 "updateStrategy":{"rollingUpdate":
 {"maxSurge":1,"maxUnavailable":0}}}}'
```

OUTPUT

```
daemonset.apps/node-agent patched
```

$ Terminal

```
kubectl set env daemonset/node-agent INTERVAL=15
kubectl get pods -l app=node-agent -o wide
```

OUTPUT

```
node-agent-5d968   lab-worker2
node-agent-d88wn   lab-worker2  ← old + new
```

$ Terminal

```
kubectl rollout status daemonset/node-agent
```

OUTPUT

```
daemon set "node-agent" successfully rolled out
```

maxSurge needs maxUnavailable: 0 and no hostPort. Step 11's maxUnavailable: 1 left each node briefly without its agent.

Common pitfalls — troubleshoot the symptom

Join fails

Expired token or discovery trust mismatch.

Create a fresh join command and verify the CA hash.

Drain hangs

DaemonSet Pods or disruption constraints.

Use --ignore-daemonsets and inspect eviction blockers.

Job stuck

Bad command, image, restartPolicy or retry budget.

Inspect Job, Pod status, events and logs.

CronJob not firing

Cron syntax / controller time-zone mismatch.

Check schedule, controller time zone and `spec.timeZone`.

DaemonSet missing

Node not eligible or selector / taint rules exclude it.

Inspect node labels, tolerations and DaemonSet selector.

Node stays unschedulable

Cordon was never reversed.

Run kubectl get nodes, then uncordon intentionally.

Module recap — five operational mental models

Scale out

Provision → token → join → verify

Cordon

No new scheduling; existing Pods stay

Drain

Evict eligible workloads safely

Uncordon

Make the node schedulable again

DaemonSet

One Pod per eligible node

Job

Run to completion; retry on failure

CronJob

Schedule Jobs repeatedly

Observe

kubectl get / describe / logs / events

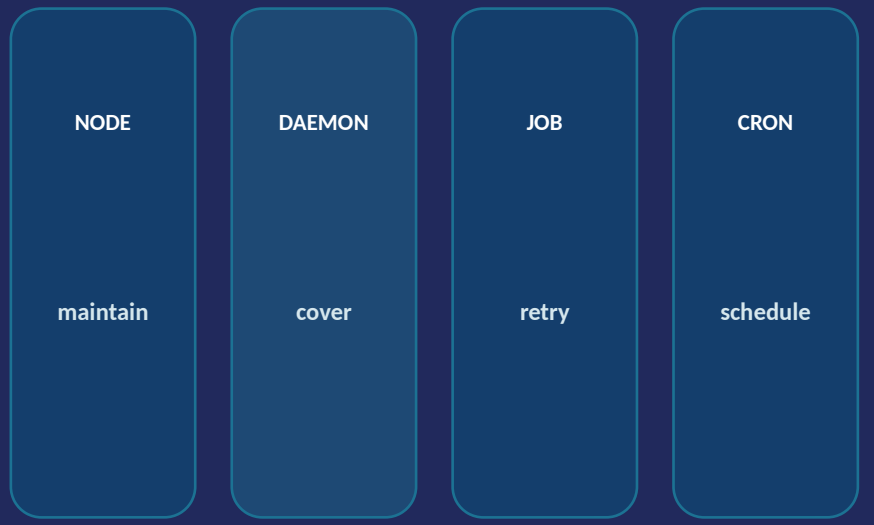
Operate the node safely first; then choose the controller that matches the workload pattern.

KNOWLEDGE CHECK

9 questions

Before you move on

Real node, DaemonSet and batch problems, with the terminal output you'd see. Answer key on the last quiz slide.



Question 1 of 9

A node drain has been retrying for 10 minutes with the message below. What is the right fix?

$ Scenario

```
k drain lab-worker --ignore-daemonsets
```

OUTPUT

```
error when evicting pods/"api-7c9-x2" (will
retry after 5s): Cannot evict pod as it
would violate the pod's disruption budget.
```

Think about

What does ALLOWED DISRUPTIONS say?

CHOOSE ONE

A. Delete the Pods with --grace-period=0
B. Let the PDB allow one disruption (e.g. maxUnavailable: 1) or add replicas
C. Restart the kubelet on the node
D. Use kubectl cordon instead of drain

Question 2 of 9

You run kubectl drain and get this error. What do you add?

$ Scenario

```
k drain worker-two
```

OUTPUT

```
cannot delete DaemonSet-managed Pods
(use --ignore-daemonsets to ignore):
kube-system/kube-proxy-x7k2p
```

Think about

Would evicting it even help?

CHOOSE ONE

A. --force
B. --delete-emptydir-data
C. --ignore-daemonsets
D. --grace-period=0

Question 3 of 9

A node is cordoned, but a new Pod still starts on it. What most likely happened?

$ Scenario

```
k get pod debug -o wide
```

OUTPUT

```
debug   1/1   Running   lab-worker

# lab-worker: Ready,SchedulingDisabled
```

Think about

Who reads the unschedulable taint?

CHOOSE ONE

A. The Pod set spec.nodeName, which bypasses the scheduler
B. Cordon only works for Deployments
C. The cordon expired after 5 minutes
D. The node was uncordoned by the kubelet

Question 4 of 9

After maintenance you uncordon the node. An hour later it still runs zero app Pods. Why?

$ Scenario

```
k get pods -o wide | grep lab-worker
```

OUTPUT

```
(nothing)
```

Think about

Scheduling happens once, at Pod creation

CHOOSE ONE

A. The node is still broken
B. Uncordon needs a kubelet restart
C. PDBs block Pods from returning
D. Kubernetes doesn't move running Pods back; new Pods (e.g. rollout restart) will use it

Question 5 of 9

Two days after the cluster was built, the join command from your notes hangs with this message. What do you do?

$ Scenario

```
kubeadm join … --token k8s123.…
```

OUTPUT

```
[discovery] Retrying due to error: could not
find a JWS signature in the cluster-info
ConfigMap for token ID "k8s123"
```

Think about

Bootstrap tokens are short-lived

CHOOSE ONE

A. Create a fresh one: kubeadm token create --print-join-command
B. Re-run kubeadm init on the control plane
C. Copy /etc/kubernetes/admin.conf to the new node
D. Add --ignore-preflight-errors=all

Question 6 of 9

A 4-node cluster (1 control plane + 3 workers). Your log-agent DaemonSet shows DESIRED 3. Why?

$ Scenario

```
k get ds log-agent
```

OUTPUT

```
NAME        DESIRED   CURRENT   READY
log-agent   3         3         3
```

Think about

Taints and tolerations

CHOOSE ONE

A. One worker is out of memory
B. DaemonSets only run on worker nodes, always
C. The control plane has a NoSchedule taint the Pod doesn't tolerate
D. The DaemonSet is still rolling out

Question 7 of 9

A Job fails with exit code 42 (bad input) and still retries 6 times over 10 minutes. How do you make it fail at once?

$ Scenario

```
k get pods -l job-name=import
```

OUTPUT

```
import-2x8k   0/1   Error   (exit 42)
import-9fqd   0/1   Error   (exit 42)
… 7 Pods
```

Think about

Not every failure deserves a retry

CHOOSE ONE

A. Set restartPolicy: Always
B. Add a podFailurePolicy rule: exit code 42 → FailJob
C. Set parallelism: 0
D. Delete the Job after the first failure by hand

Question 8 of 9

A CronJob with schedule "0 9 * * *" should run at 09:00 in Berlin, but it runs at 11:00 Berlin time. Fix?

$ Scenario

```
k get cronjob daily-report
```

OUTPUT

```
SCHEDULE    TIMEZONE   LAST SCHEDULE
0 9 * * *   <none>     …
```

Think about

Whose 09:00 is it?

CHOOSE ONE

A. Change the schedule to 0 7 * * *
B. Restart kube-controller-manager
C. Use a Job instead
D. Set spec.timeZone: "Europe/Berlin"

Question 9 of 9

A nightly backup CronJob sometimes takes longer than a day, and two backups then run at once. What prevents that?

$ Scenario

```
k get jobs
```

OUTPUT

```
backup-29844000   Running   25h
backup-29845440   Running   1h
```

Think about

Overlap is a CronJob setting

CHOOSE ONE

A. successfulJobsHistoryLimit: 1
B. backoffLimit: 0
C. concurrencyPolicy: Forbid
D. parallelism: 1

Answer key

Score yourself

9 questions
1 point each

ANSWERS

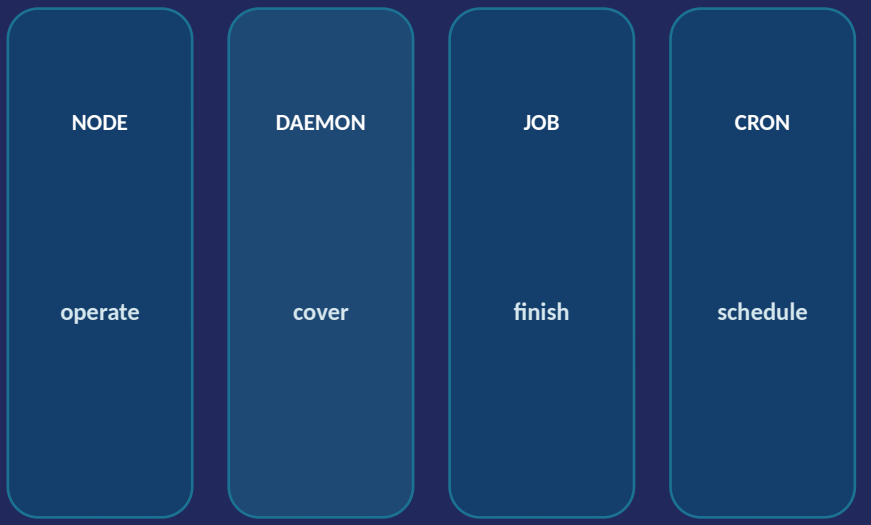
Q1	B	let the PDB allow 1 disruption	→ slide 39 • Step 6
Q2	C	--ignore-daemonsets	→ slide 14 • Step 5
Q3	A	nodeName skips the scheduler	→ slide 38 • Step 4
Q4	D	Pods don't move back	→ slide 40 • Step 7
Q5	A	create a fresh join command	→ slide 7 • Step 2
Q6	C	control-plane taint	→ slide 41 • Step 9
Q7	B	podFailurePolicy FailJob	→ slide 42 • Step 14
Q8	D	spec.timeZone	→ slide 33 • Step 16
Q9	C	concurrencyPolicy: Forbid	→ slide 43 • Step 16

8–9	Module 6 ready
6–7	re-read misses
≤ 5	redo Steps 4–16

MODULE 5 COMPLETE

Next: Module 6

ConfigMaps, Secrets, Init Containers & Sidecars



The node is now an operational surface: scale it, maintain it, run node-local agents, and schedule finite work.